



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Advanced Database Systems (INFS3200)

Lecture 12: Data-Driven Applications

Lecturer: Dr Zhi Chen

Data Lake: Quick Recap

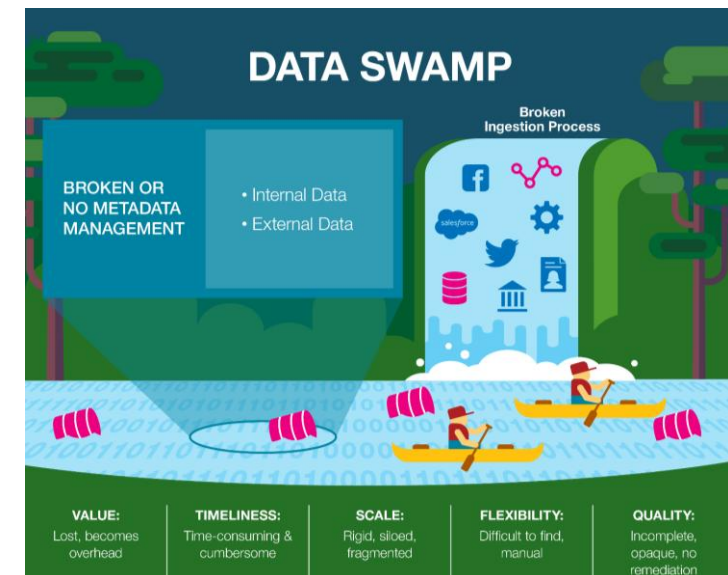
Data lake:

- Ingests and stores raw data from heterogeneous sources in their original format
- Advantages in big data era: schema-on-read, ELT ...

Data Lake: Quick Recap

“Data lake” or “Data swamp”:

- Ingestion and maintenance
- Ingestion: metadata extraction, metadata modelling
- Maintenance: dataset organization, data integration, data cleaning, metadata enrichment, data provenance, related dataset discovery, schema evolution.



Data Lake: Quick Recap

From data lake to data lakehouse:

- Aims to tackle four challenges:
 - **Reliability**
 - **Data staleness**
 - **Limited support for advanced analytics**
 - **Total cost of ownership**

Outline

- **Big Data** Applications
 - Connecting Dots: **From Small to Big**
 - Discovering Specifics: **From Big to Small**
 - Inferencing: **Knowing Unknown**
 - **Dimensionality: Blessing or Curse?**
- **Effective Storage** of Big Data
 - **SQL or NoSQL**
 - **Vector database**

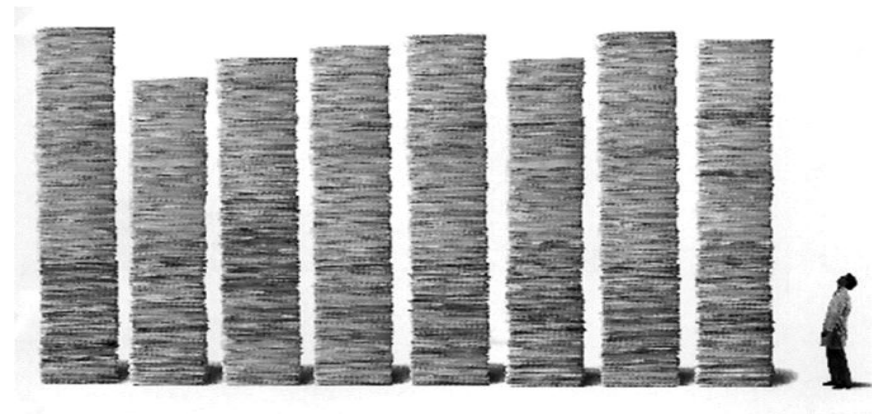
How do we use data?

Round 1					Totals					Shooting			Per Game				Advanced				
Rk	Pk	Tm	Player	College	Yrs	G	MP	PTS	TRB	AST	FG%	3P%	FT%	MP	PTS	TRB	AST	WS	WS/48	BPM	VORP
1	1	PHI	Ben Simmons	Louisiana State University																	
2	2	LAL	Brandon Ingram	Duke University	1	79	2279	740	317	166	.402	.294	.621	28.8	9.4	4.0	2.1	-0.3	-.007	-3.8	-1.1
3	3	BOS	Jaylen Brown	University of California	1	78	1341	515	220	64	.454	.341	.685	17.2	6.6	2.8	0.8	1.5	.053	-4.0	-0.7
4	4	PHO	Dragan Bender		1	43	574	146	103	23	.354	.277	.364	13.3	3.4	2.4	0.5	-0.3	-.029	-4.3	-0.3
5	5	MIN	Kris Dunn	Providence College	1	78	1333	293	166	188	.377	.288	.610	17.1	3.8	2.1	2.4	0.1	.004	-2.2	-0.1
6	6	NOP	Buddy Hield	University of Oklahoma	1	82	1888	866	269	121	.426	.391	.842	23.0	10.6	3.3	1.5	1.3	.032	-2.8	-0.4
7	7	DEN	Jamal Murray	University of Kentucky	1	82	1764	811	214	170	.404	.334	.883	21.5	9.9	2.6	2.1	1.4	.037	-2.6	-0.3
8	8	SAC	Marquese Chriss	University of Washington	1	82	1743	753	348	60	.449	.321	.624	21.3	9.2	4.2	0.7	1.8	.050	-1.6	0.2
9	9	TOR	Jakob Poeltl	University of Utah	1	54	626	165	165	12	.583		.544	11.6	3.1	3.1	0.2	1.6	.125	-0.7	0.2
10	10	MIL	Thon Maker		1	57	562	226	114	23	.459	.378	.653	9.9	4.0	2.0	0.4	1.3	.113	-1.5	0.1
11	11	ORL	Domantas Sabonis	Gonzaga University	1	81	1632	479	287	82	.399	.321	.657	20.1	5.9	3.5	1.0	0.7	.022	-4.9	-1.2
...																					
27	27	TOR	Pascal Siakam	New Mexico State University	1	55	859	229	185	17	.502	.143	.688	15.6	4.2	3.4	0.3	1.1	.031	-1.1	-0.1
28	28	PHO	Skal Labissiere	University of Kentucky	1	33	612	289	162	27	.537	.375	.703	18.5	8.8	4.9	0.8	1.1	.031	-1.1	-0.1
29	29	SAS	Dejounte Murray	University of Washington	1	38	322	130	42	48	.431	.391	.700	8.5	3.4	1.1	1.1	0.0	.000	-0.7	-0.1
30	30	GSW	Damian Jones	Vanderbilt University	1	10	85	19	23	0	.500		.300	8.5	1.9	2.3	0.0	0.0	.000	-0.7	-0.1
...																					
Round 2					Totals					Shooting			Per Game				Advanced				
Rk	Pk	Tm	Player	College	Yrs	G	MP	PTS	TRB	AST	FG%	3P%	FT%	MP	PTS	TRB	AST	WS	WS/48	BPM	VORP
31	31	BOS	Devyonta Davis	Michigan State University	1	36	238	58	60	2	.511		.556	6.6	1.6	1.7	0.2	0.0	.000	-0.7	-0.1
32	32	LAL	Ivica Zubac		1	38	609	284	159	30	.529	.000	.653	16.0	7.5	4.2	0.8	1.1	.031	-1.1	-0.1
33	33	LAC	Cheick Diallo	University of Kansas	1	17	199	87	73	4	.474		.714	11.7	5.1	4.3	0.2	0.0	.000	-0.7	-0.1
34	34	PHO	Tyler Ulis	University of Kentucky	1	61	1123	444	95	226	.421	.266	.775	18.4	7.3	1.6	3.7	0.0	.000	-0.7	-0.1
35	35	BOS	Rade Zagorac																		
36	36	MIL	Malcolm Brogdon	University of Virginia	1	75	1982	767	213	317	.457	.404	.865	26.4	10.2	2.8	4.2	0.0	.000	-0.7	-0.1
...																					

How do we make sense of Big Data?

Big data is to solve three problems:

1. Problem of *from-small-to-big* (Connecting Dots)
2. Problem of *from-big-to-small* (Discovering Specifics)
3. Problem of “*knowing unknown*” – everything is related to everything else (Inferencing)



3-1. Connecting Dots:

Big data is a problem of “from-small-to-big”

Given a piece of data, how do we find the ones that are relevant to it?



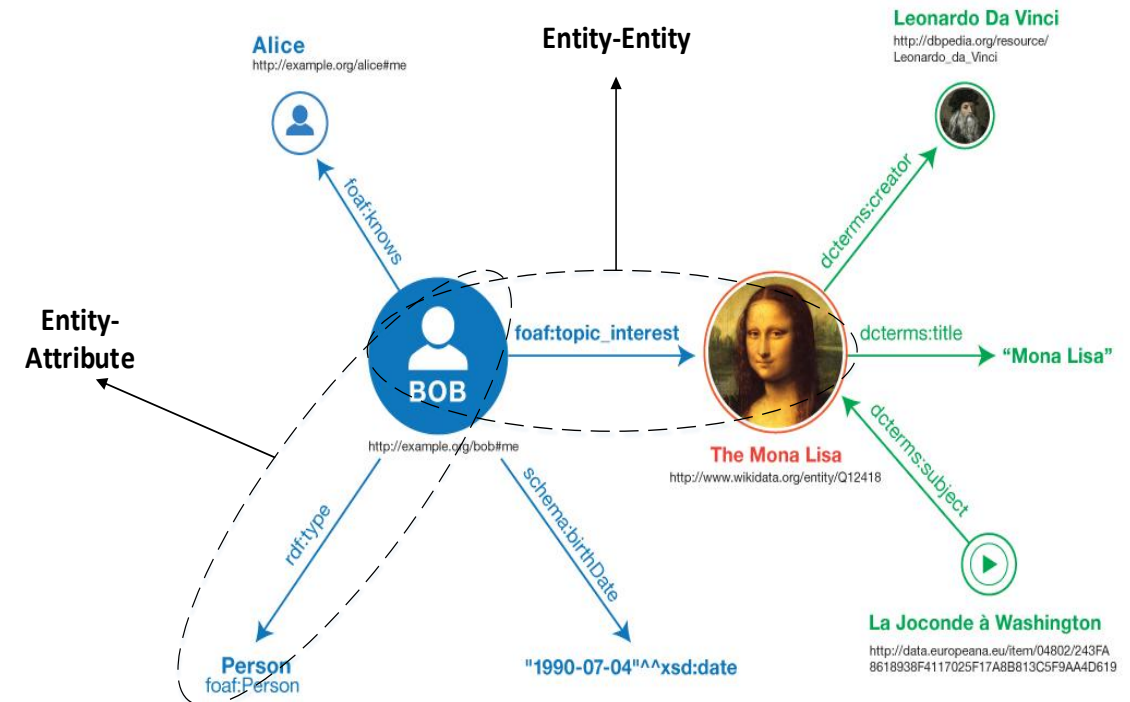
Aristotle: The whole is more than the sum of its parts.

Key Value – The smallest semantic unit of Big Data

- [key, value] is a minimum semantic unit representing an elementary fact.
- Every [key value] is unique with a timestamp.
- Key values are associated to each other.
- All key values can be drawn as a graph.

RDF triples:

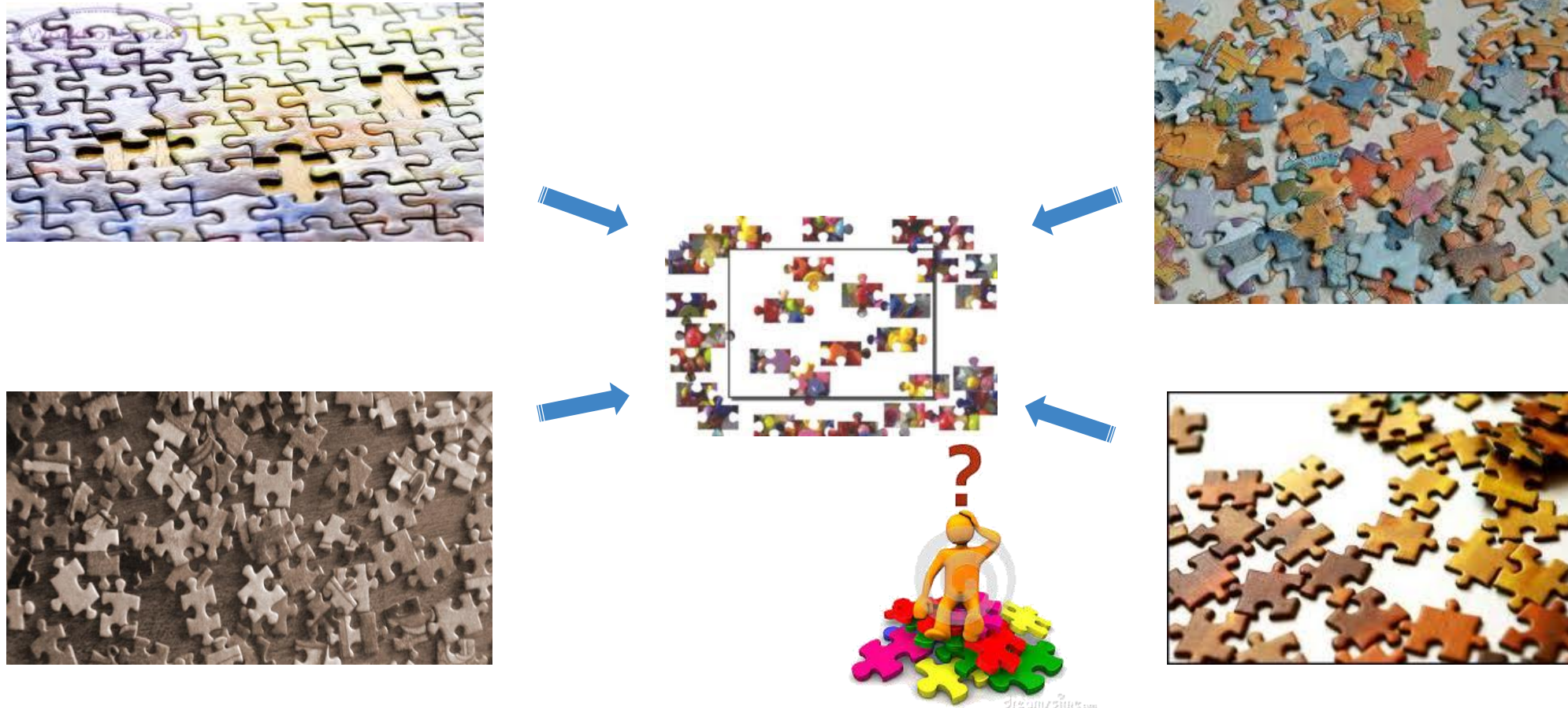
Bob	type	Person.
bob	knows	alice.
bob	birthDate	"1990-07-04".
bob	topic_interest	The Mona Lisa.
The Mona Lisa	creator	Leonardo_da_Vinci .
La Joncode a Wa.	subject	The Mona Lisa.



Key-Value Stores

- A simple user interface
 - Data model: (key, value) pairs
 - Operations: Insert(key, value), Fetch(key), Update(key), Delete(key)
 - Some allow (non-uniform) columns within value
 - Some allow Fetch on range of keys
- Example systems
 - Google BigTable, Amazon Dynamo, Cassandra, Voldemort, HBase, ...

From-small-to-big: Key Values in Unstructured Data



If every piece of value is represented by a unique key, all values can be connected together.

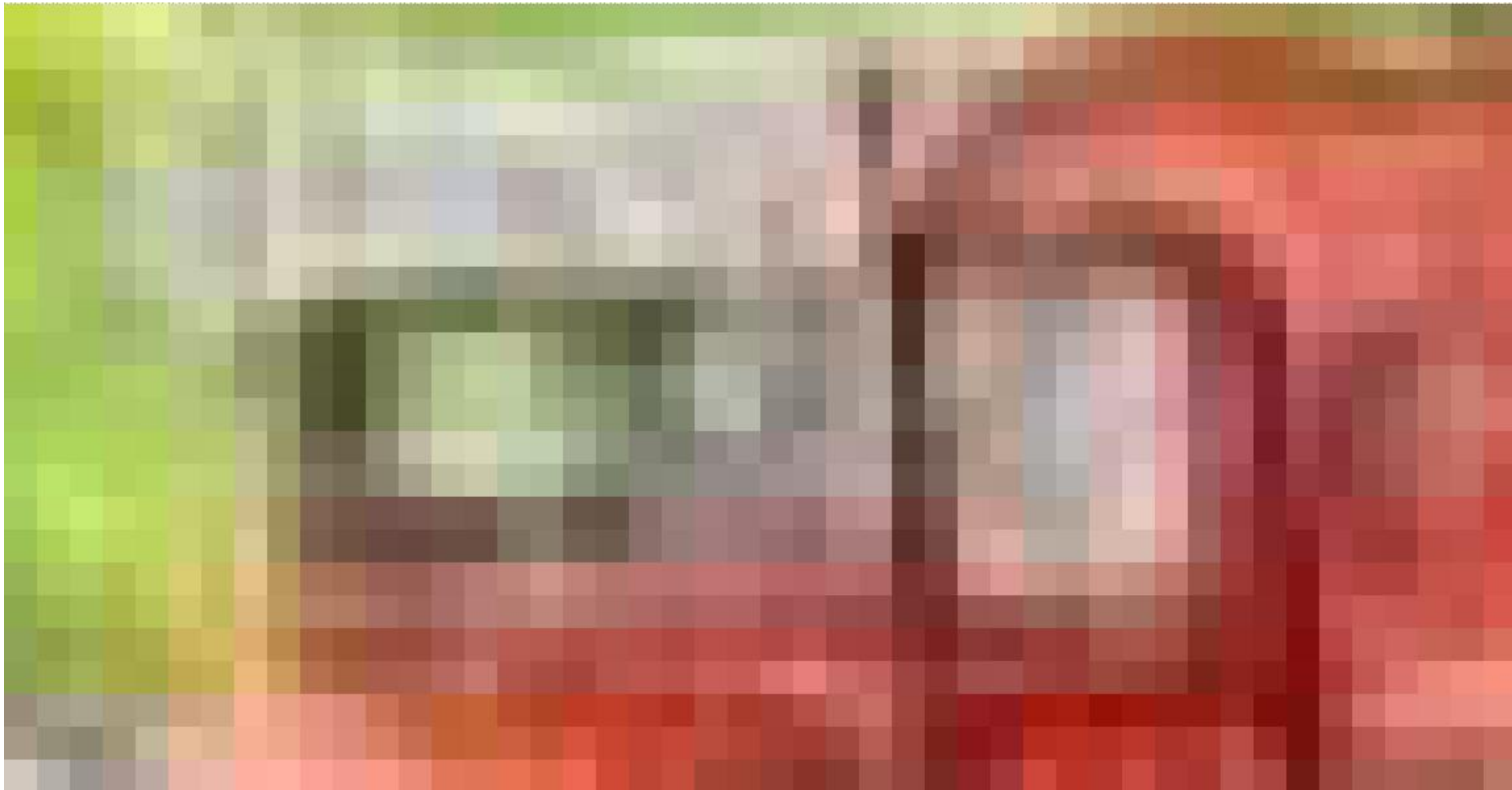
What do you get in pixels of a picture?

00101001	10101001	11100011	00011110	11100001	10001111	01010111	01010001	11100011	10101001
01010111	01010001	11100011	01010111	01010111	01010001	11100011	01010111	00010001	01010001
00001110	00101000	10010010	00001110	11100011	10100011	00010101	00001110	10110000	00101000
11001100	10100011	00110011	01010101	11100101	01010111	01010111	11100011	10100011	10100011
11100011	10100011	11100000	10101001	11010101	00001110	10101001	01010001	00010101	01010111
00110011	01001100	11100010	01010001	11100011	10100011	01010001	00101000	11000011	00001110
01010111	01010001	11100011	00101000	11100011	10100011	00101000	10100011	01010111	10101010
01010111	01010001	11100011	10100011	11100011	10100011	10100011	11011011	00001110	00101011

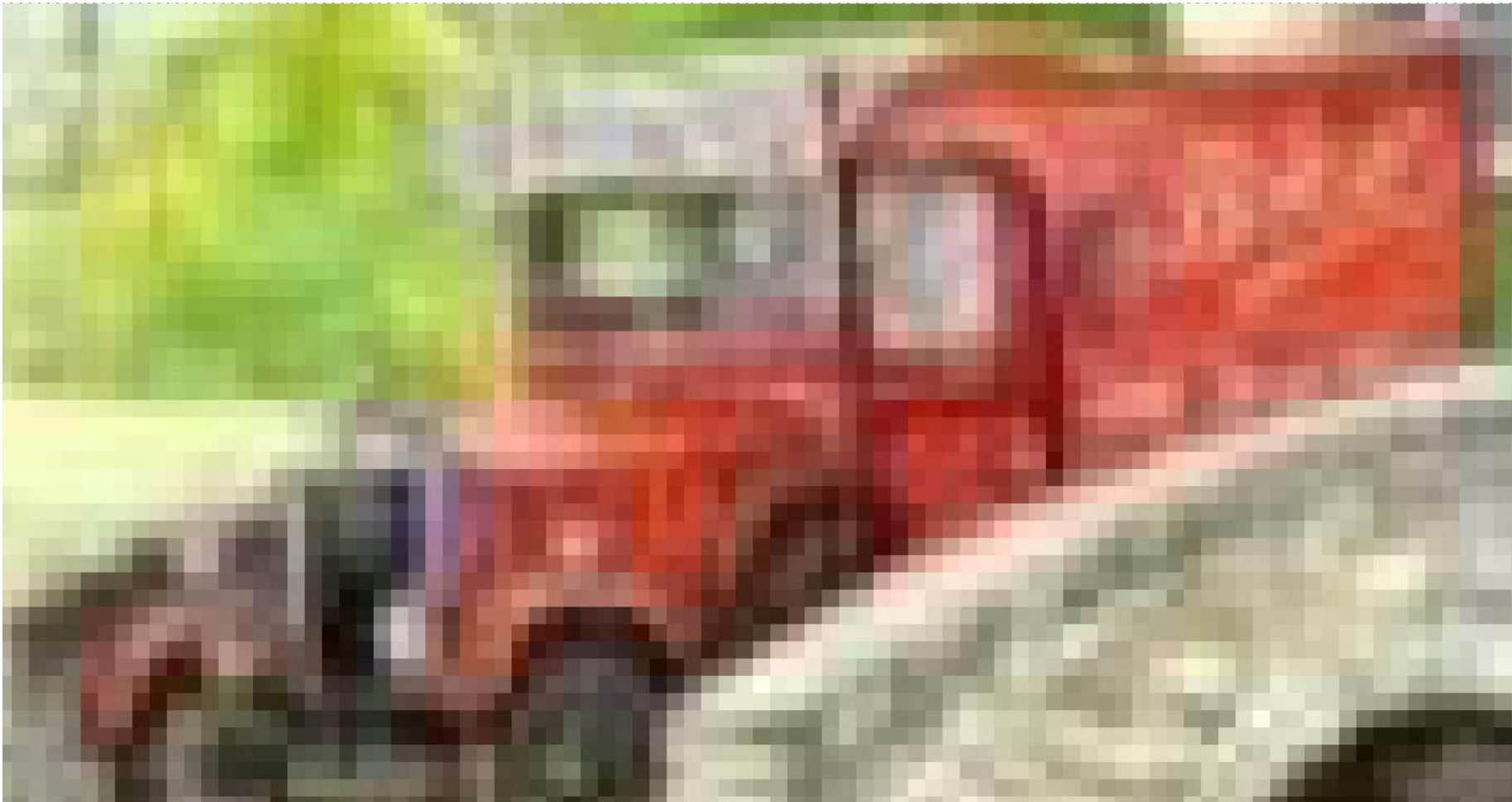
What can you see?



What can you really see?



What can you really see?



From-small-to-big: Key Values in Unstructured Data



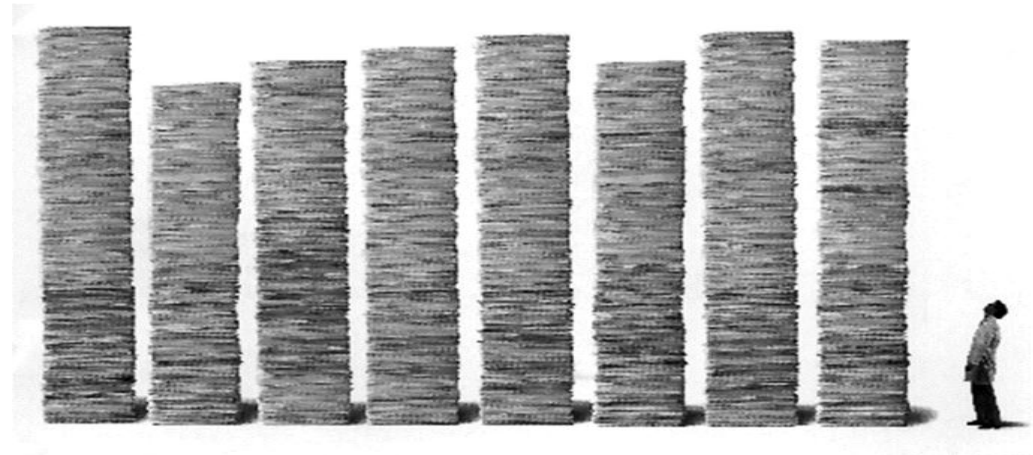
People can only see what they want to see



How do we make sense of Big Data?

Big data is to solve three problems:

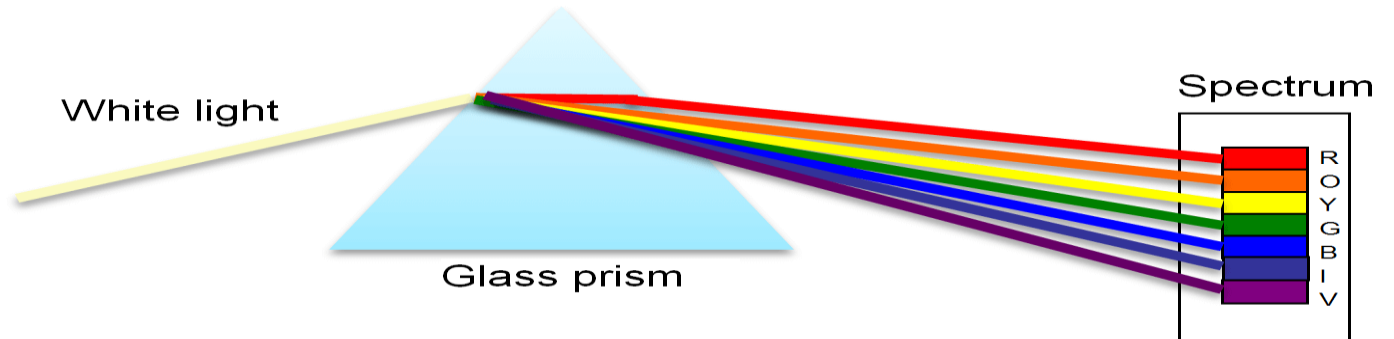
- Problem of from-small-to-big (Connecting Dots)
- **Problem of from-big-to-small** (Discovering Specifics)
- Problem of “*knowing unknown*” – everything is related to everything else (Inferencing)



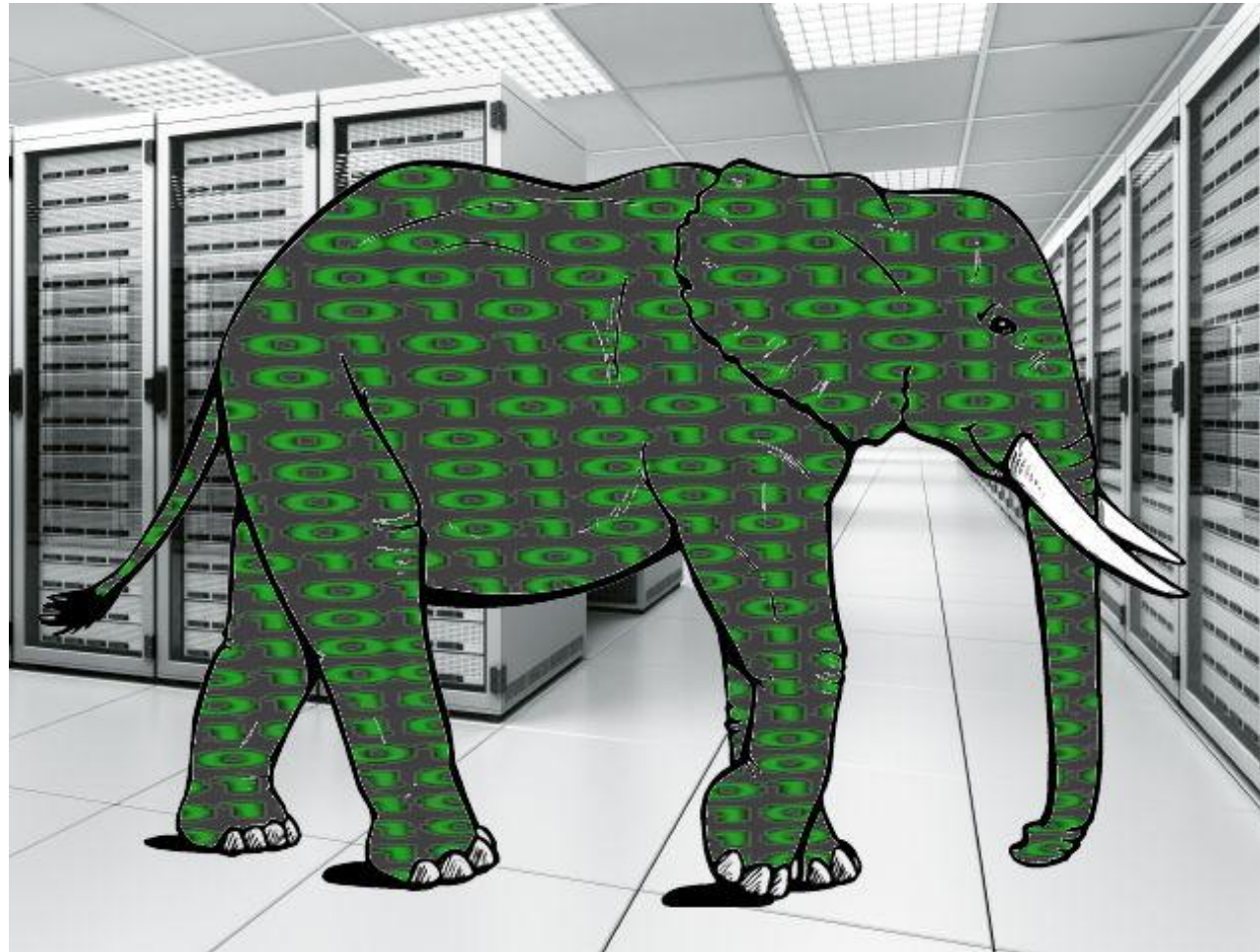
3-2. Discovering Specifics:

Big data is a problem of “from-big-to-small”

- Given a data set, how do we find outliers, predict trends, provide summaries, and give explanations?



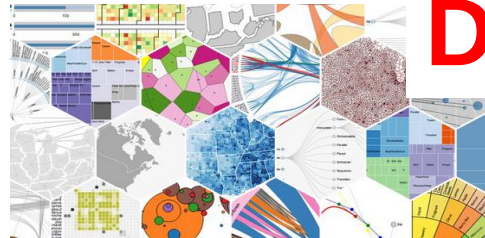
From Big-to-small: see details from a global picture



Dashboard Technology

Examples

- Opinion Search Engine
- Online learning System,
- UQ-AI-ICU System



D3



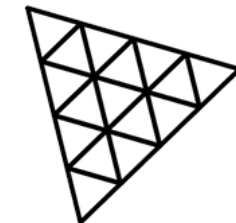
One framework. One
codebase. Any platform.
<https://www.qt.io/>

Dashboard development tools (*Django* *Vue*, *Qt*, ...)

- Layout Scheme: *Bootstrap*
- 2D-graph: *ECharts*, 3D-tree: *ThreeJS*, *D3*, ...
- *Neo4J*, *Tableau*, *Power BI*, *Qlik*, ...

Features of Dashboard Technology

- Self explanatory
- Real-time & correlated Drivers
- Drill-down hierarchical structures
- Integrated Components

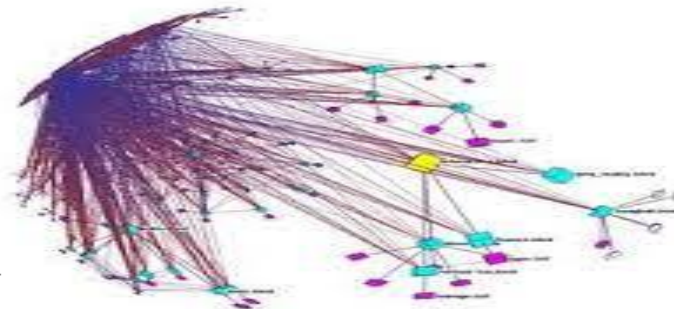


three.js



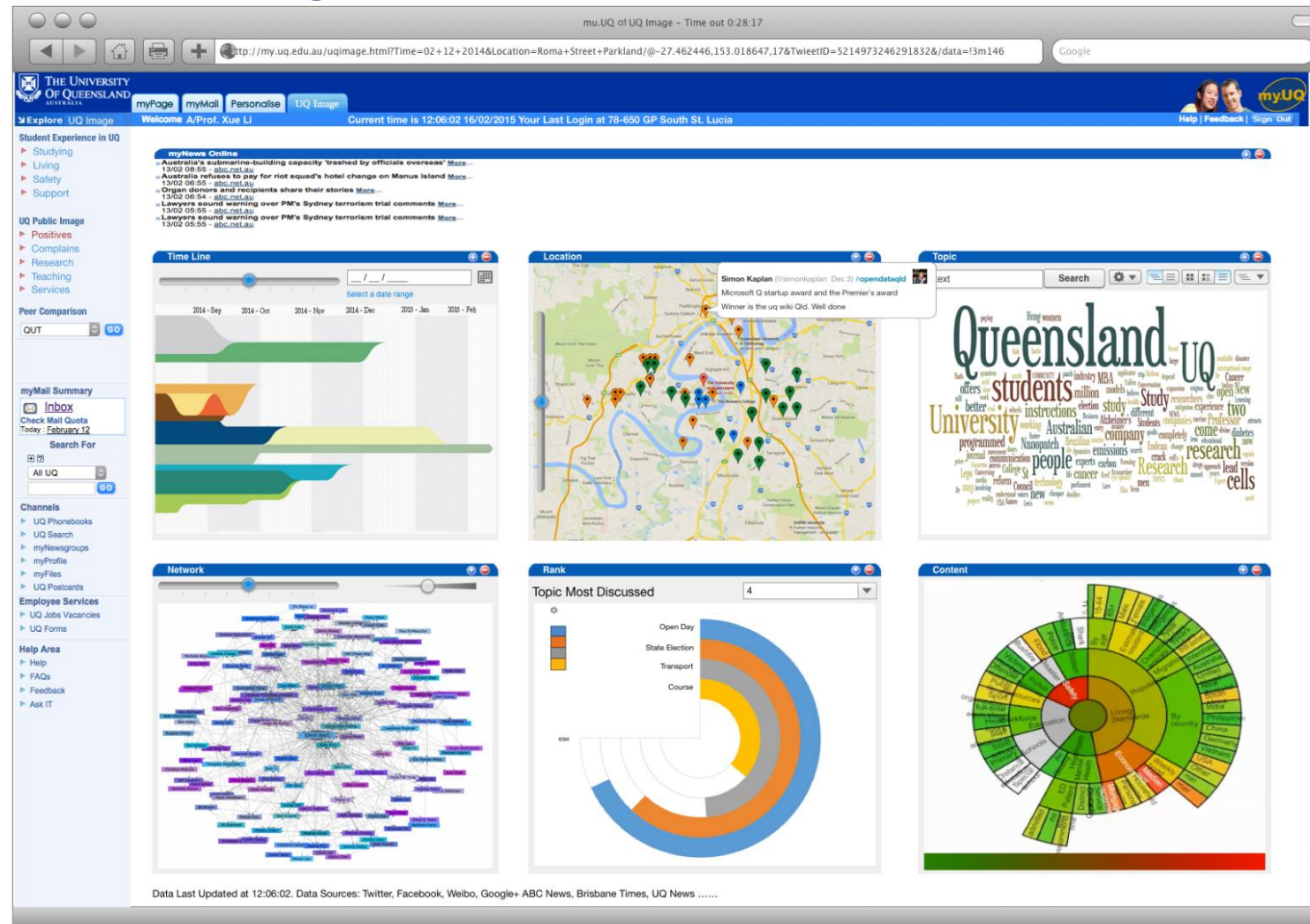
Apache ECharts

An Open Source JavaScript Visualization Library

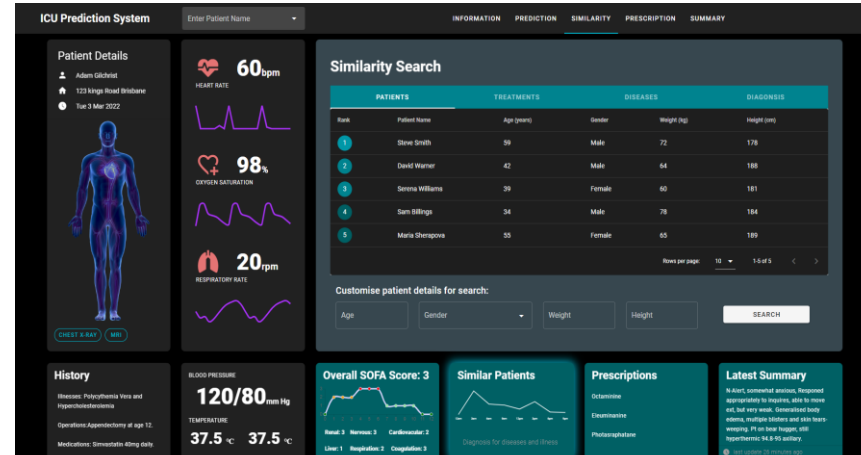
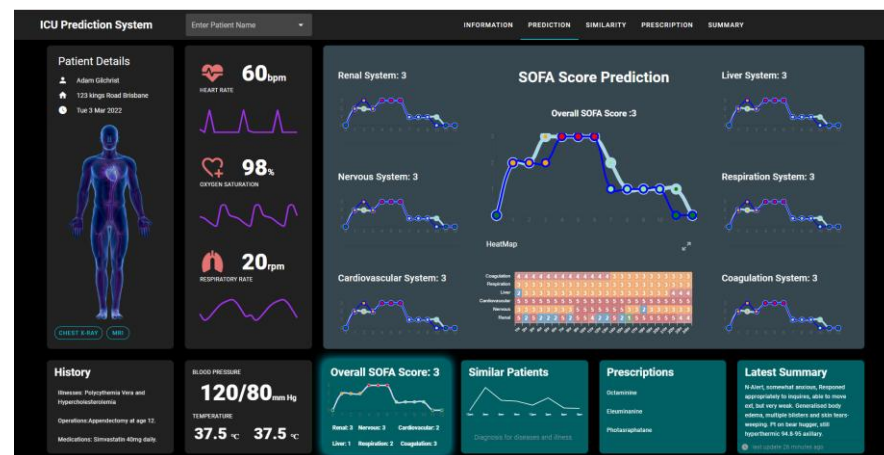
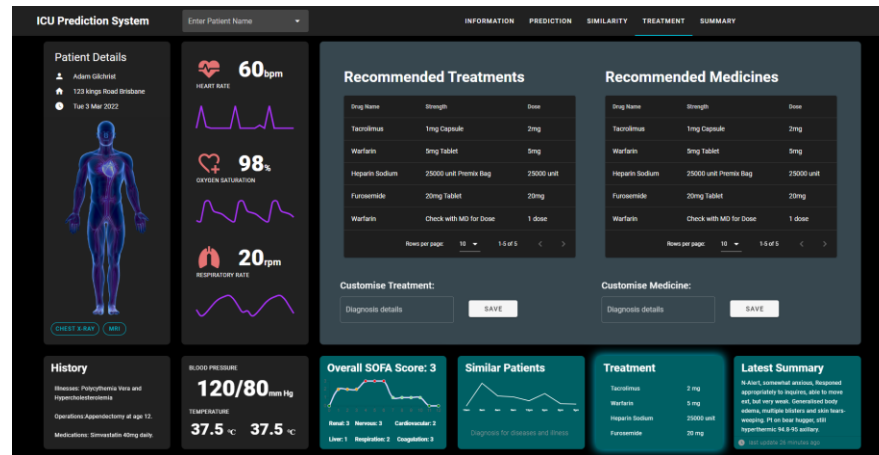
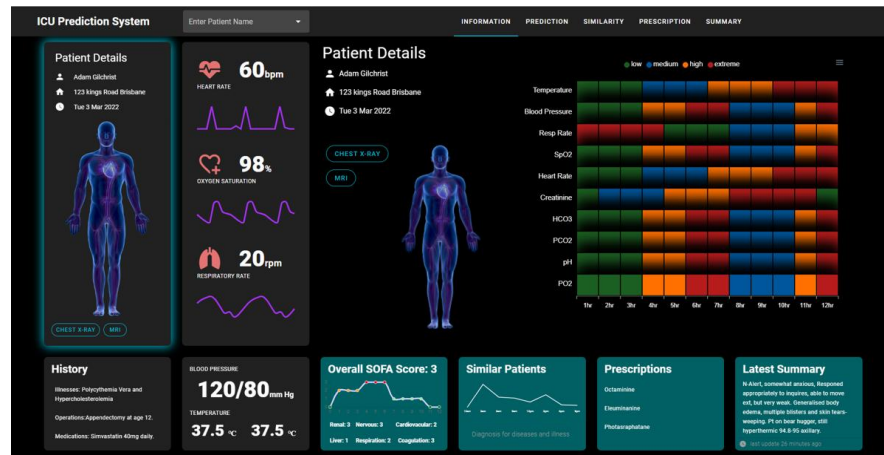


<https://getbootstrap.com/>

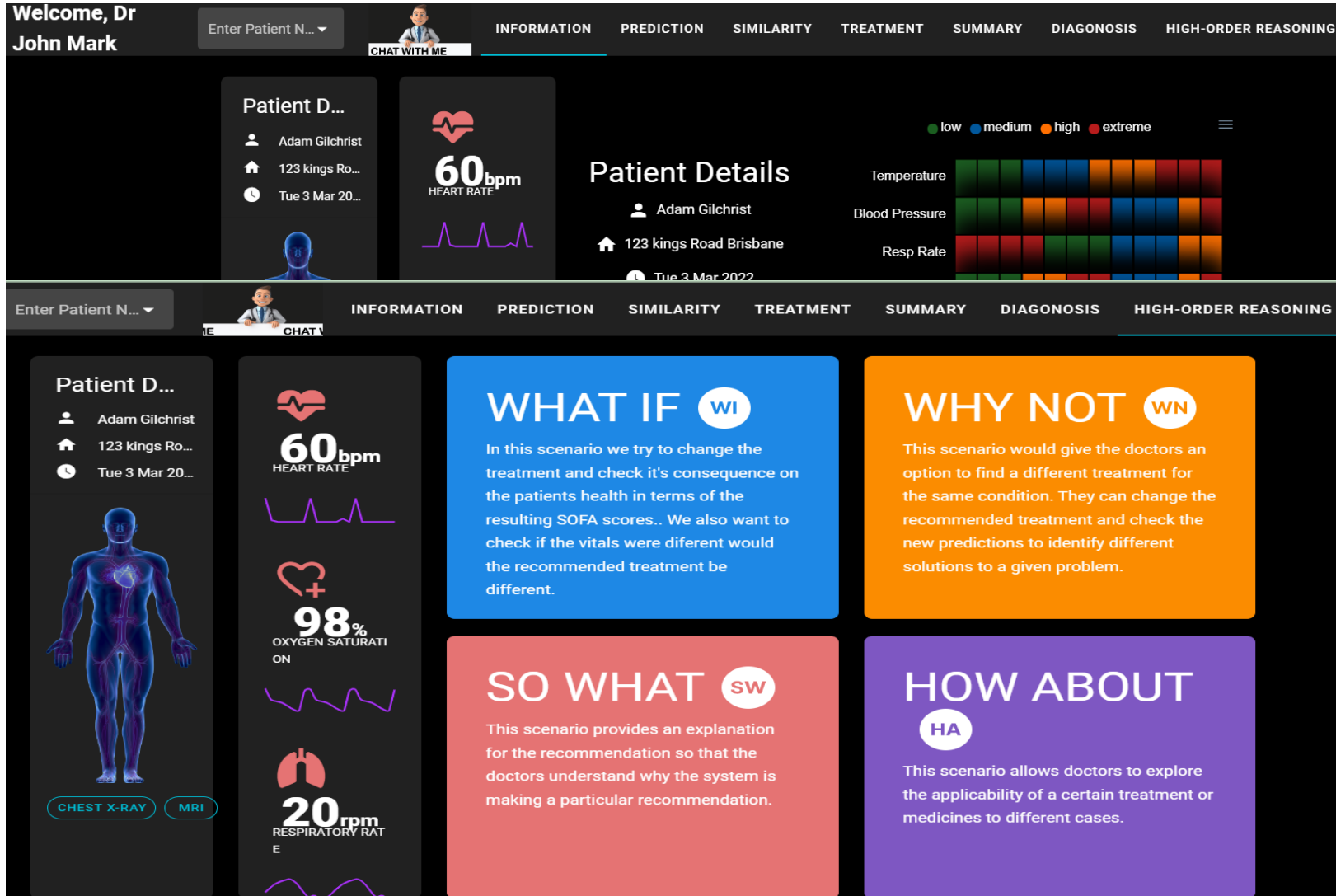
Opinion Search Engine



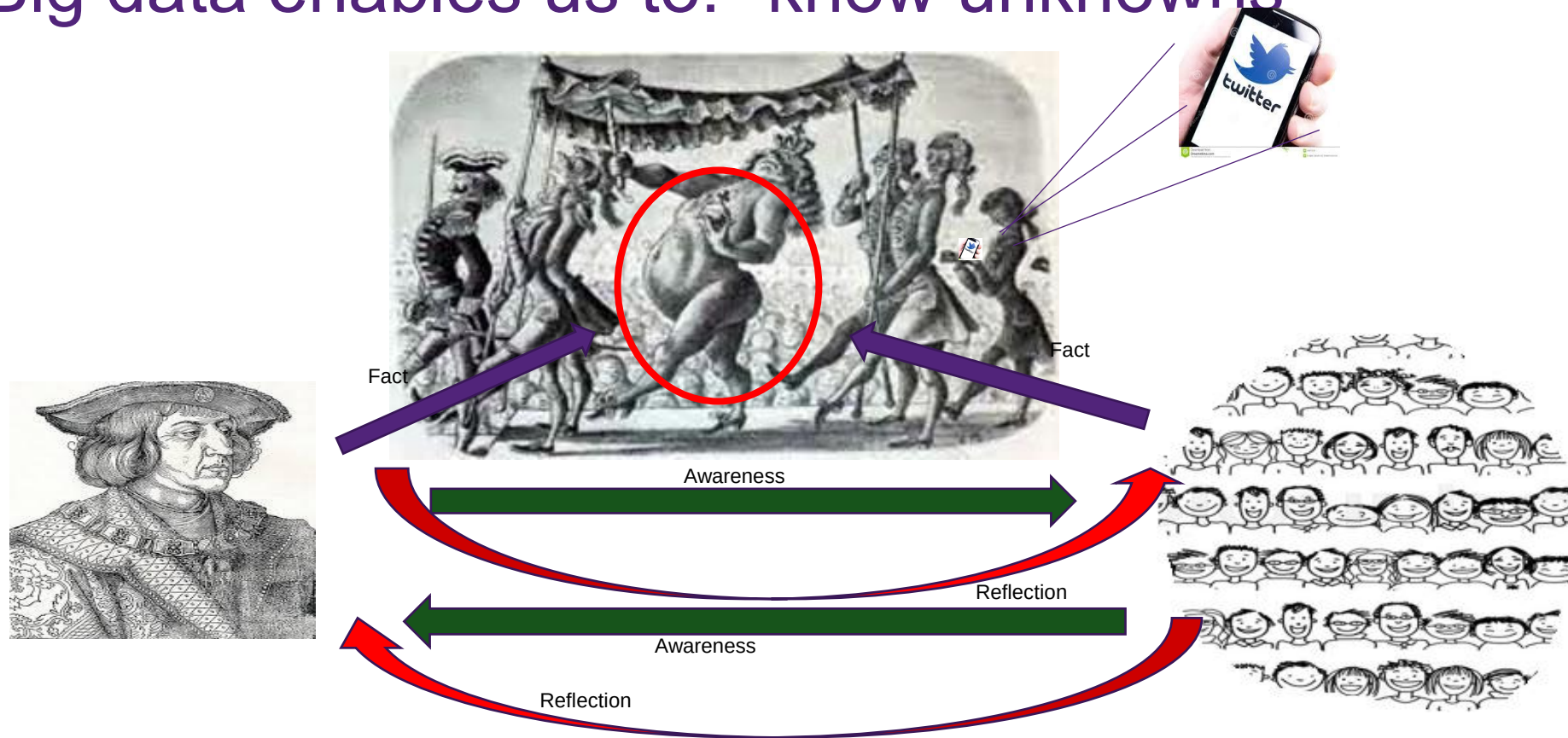
Monitoring, Prediction, Recommendation, & Search



Automatic Summarization of Patient Situation



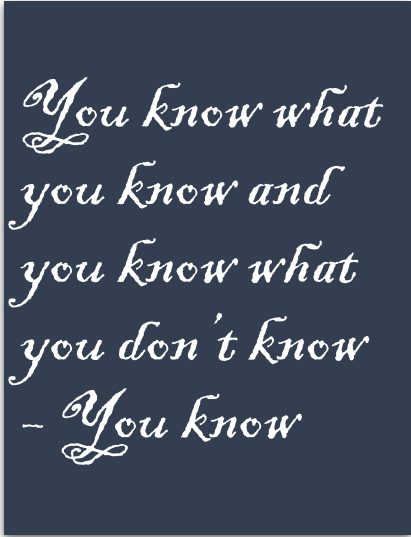
3-3. Big data enables us to: "know unknowns"



Information Island is caused by information blocking

Information Prosperity with Big Data

- 1.Fact: Does the Emperor know he wore nothing?
- 2.Fact: Do people know the Emperor wore nothing?
- 3.Awareness: Does the Emperor know people know he wore nothing?
- 4.Awareness: Do people know the Emperor knows he wore nothing?
- 5.Reflection: Does the Emperor know people know he knows he wore nothing?
- 6.Reflection: Do people know the emperor know people know he wore nothing?



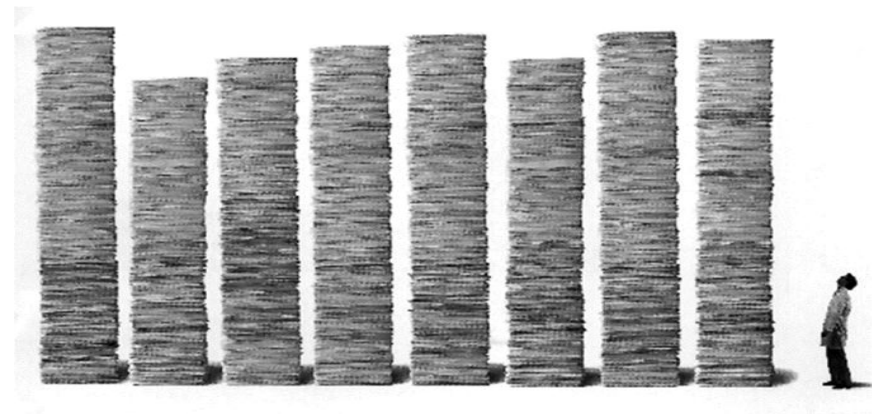
*You know what
you know and
you know what
you don't know
- You know*

Imbalanced information is the source of all problems!

How do we make sense of Big Data?

Big data is to solve three problems:

1. Problem of *from-small-to-big* (Connecting Dots)
2. Problem of *from-big-to-small* (Discovering Specifics)
3. Problem of “*knowing unknown*” – everything is related to everything else (Inferencing)



Dimensionality: Blessing or Curse?

Considering the following classification problem:

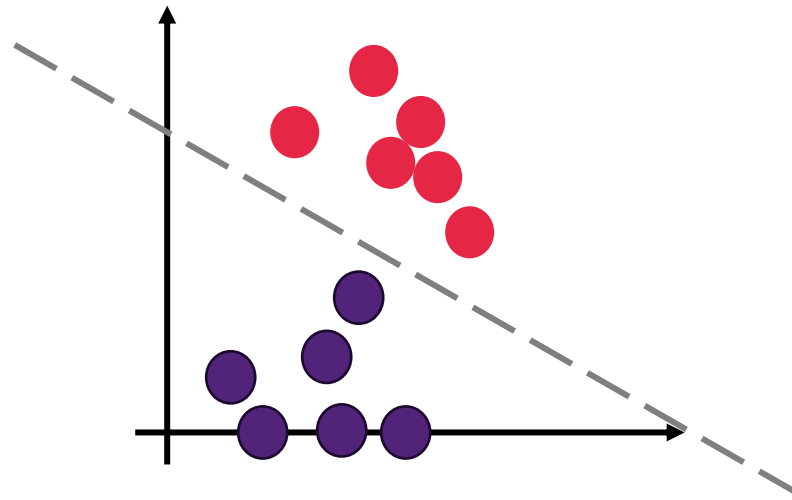
- 1-dimensional space: Linearly inseparable



Dimensionality: Blessing or Curse?

Considering the following classification problem:

- 2-dimensional space: Linearly Separable



Dimensionality: Blessing or Curse?

Increased Discriminability:

- More dimensions might allow for better separation of classes in classification problems

Richer Representations:

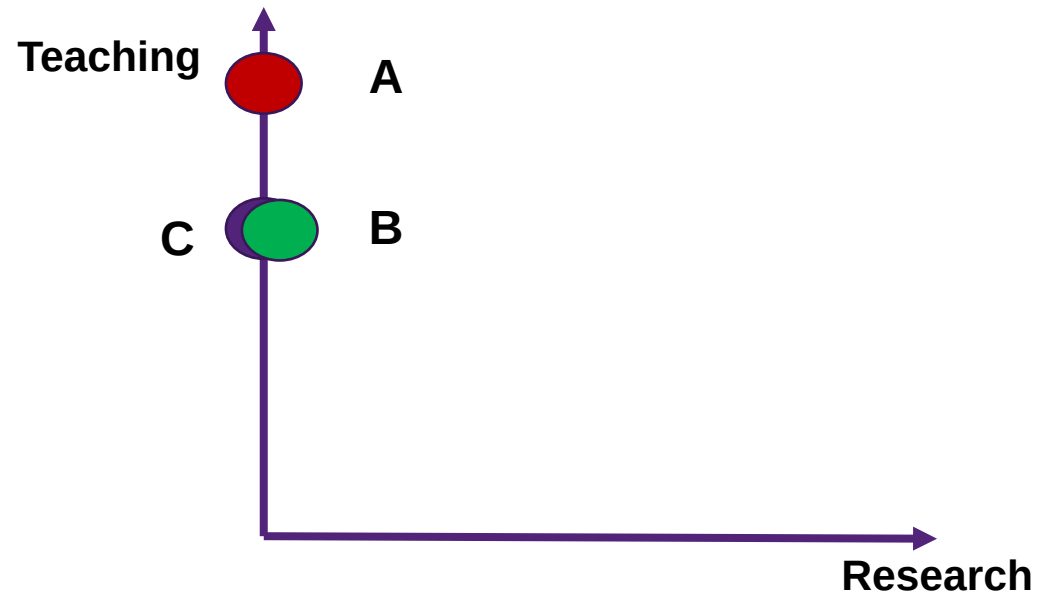
- High-dimensional representations can capture more complex structures and nuances in data.

Feature Engineering:

- Adding derived features (which increases dimensionality) can sometimes enhance model performance

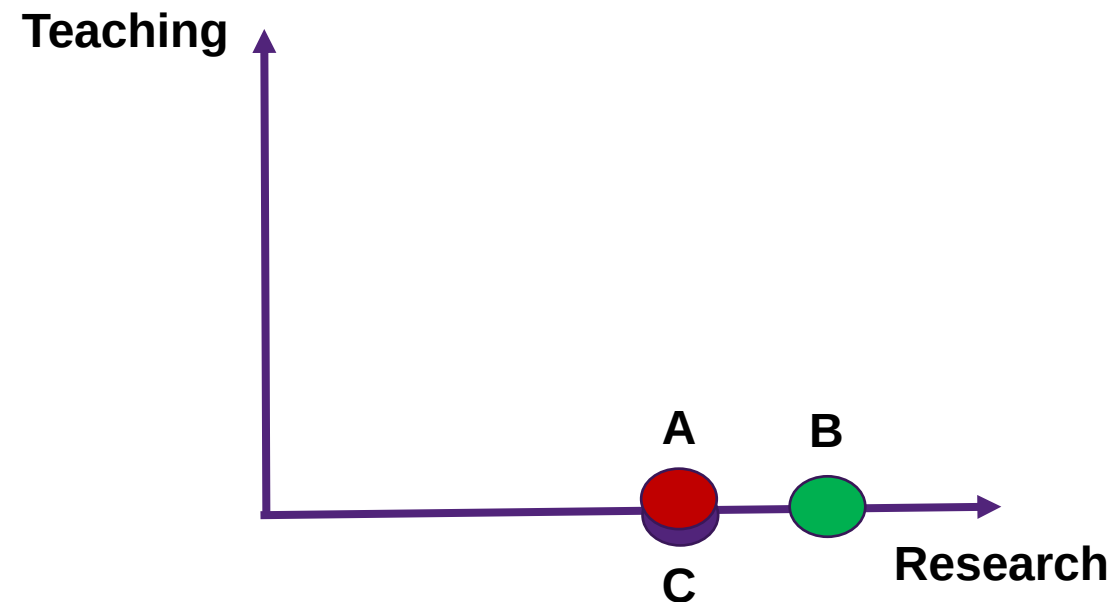
Should the dimensionality be high or Low?

Example: Given the following scores for three candidates (A, B, C) for a position of a Faculty Position in a university, which one of them should be selected to offer a job?



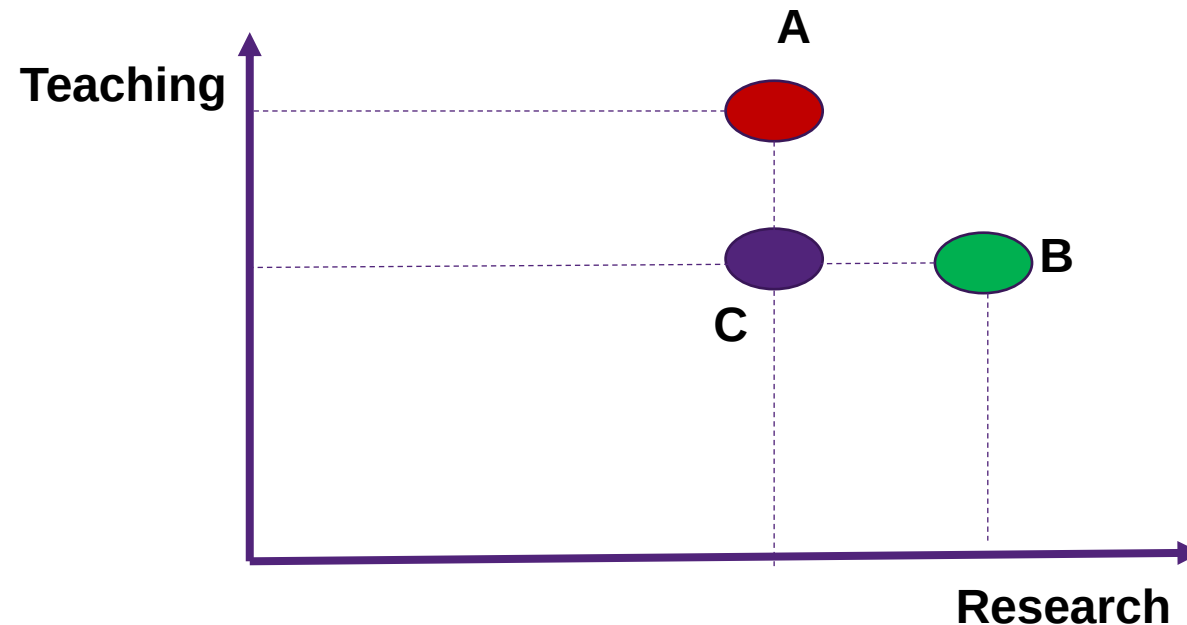
Should the dimensionality be high or Low?

Example: Given the following scores for three candidates (A, B, C) for a position of a Faculty Position in a university, which one of them should be selected to offer a job?



Should the dimensionality be high or Low?

Example: Given the following scores for three candidates (A, B, C) for a position of a Faculty Position in a university, which one of them should be selected to offer a job?



Dimensionality: Blessing or Curse?

Increased Discriminability:

- More dimensions might allow for better separation of classes in classification problems

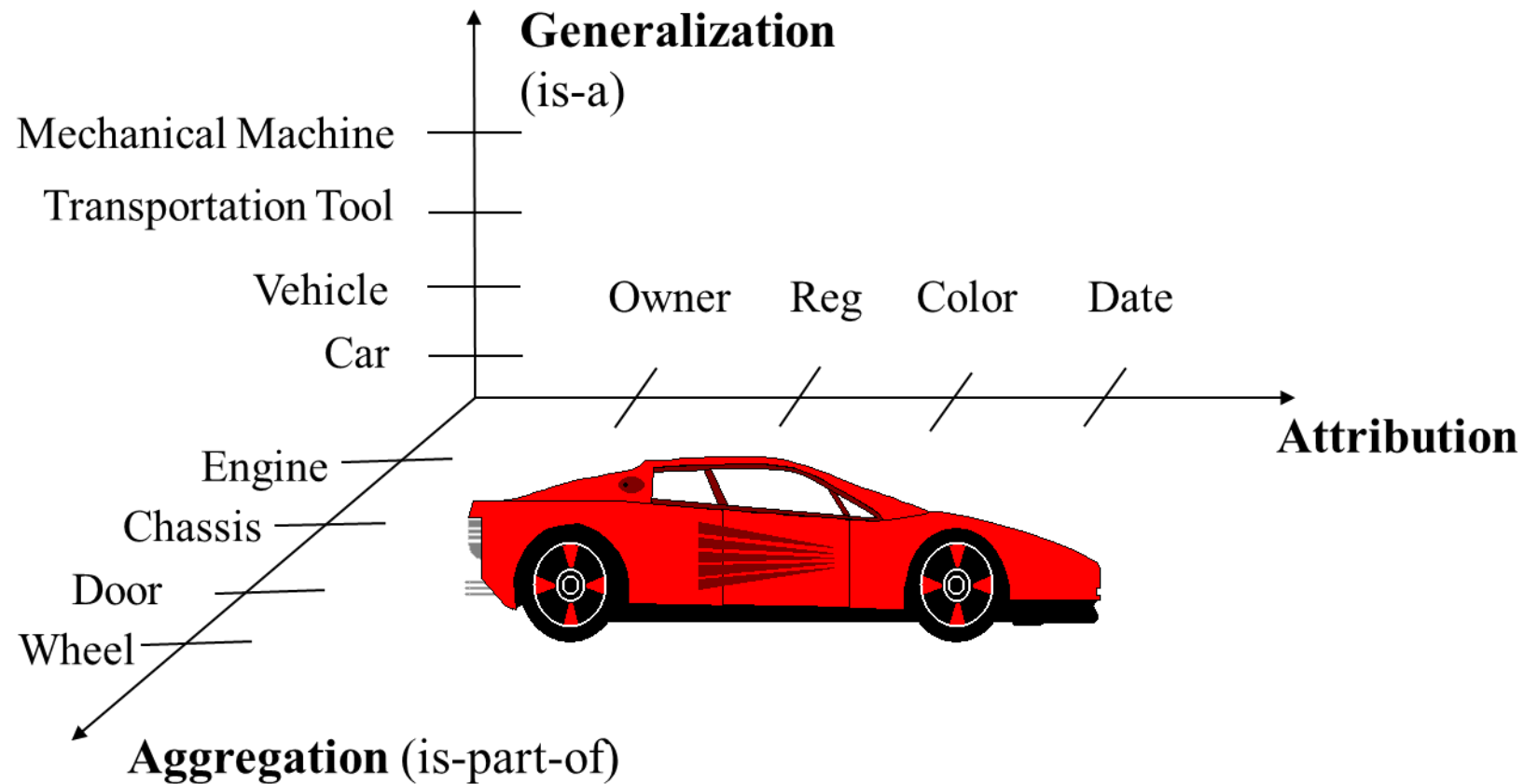
Richer Representations:

- High-dimensional representations can capture more complex structures and nuances in data.

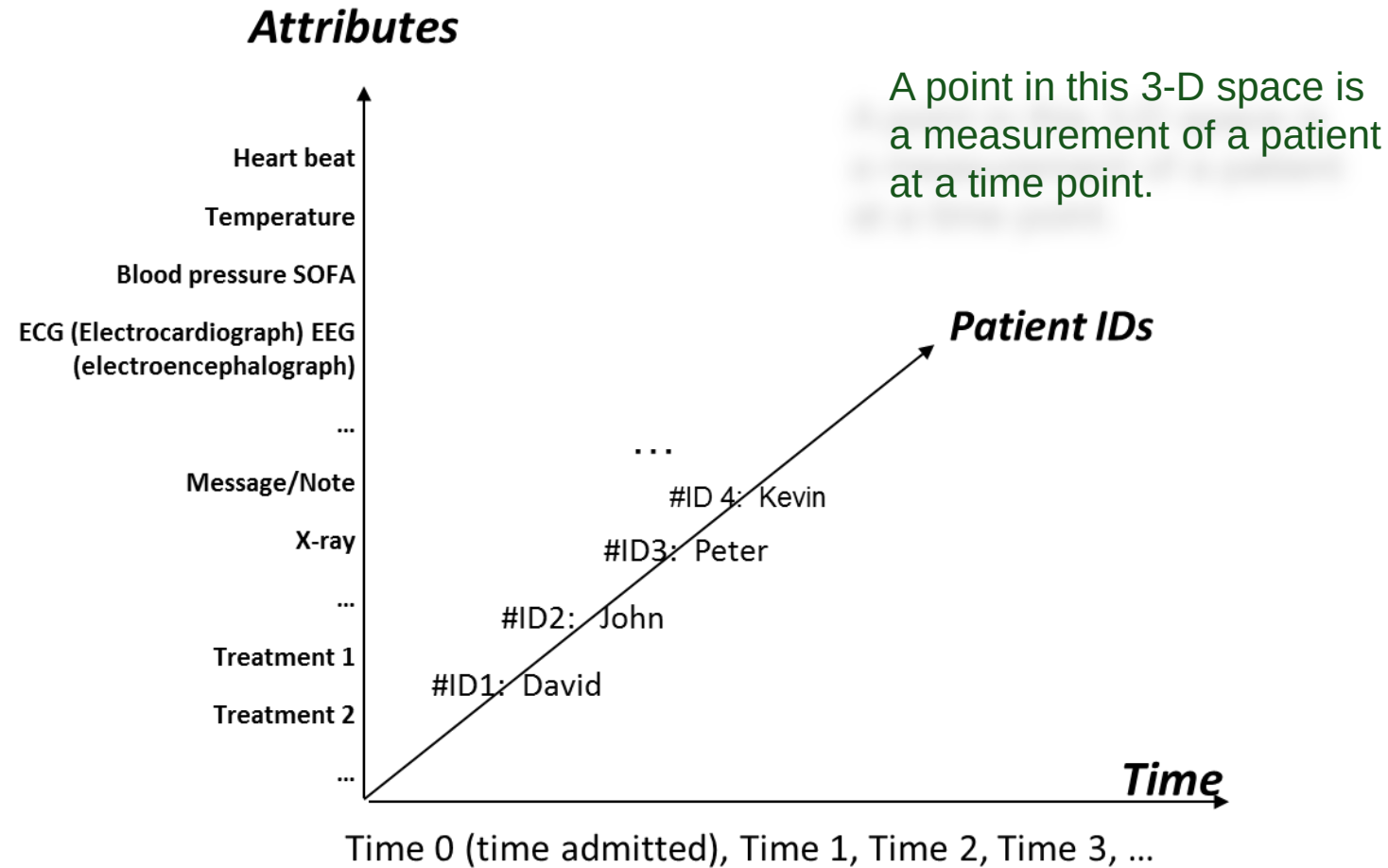
Feature Engineering:

- Adding derived features (which increases dimensionality) can sometimes enhance model performance

Example: Data for a multidimensional entity from an Object-Oriented Perspective



Example: Data for a multidimensional patient data space



Dimensionality: Blessing or Curse?

Volume Explosion:

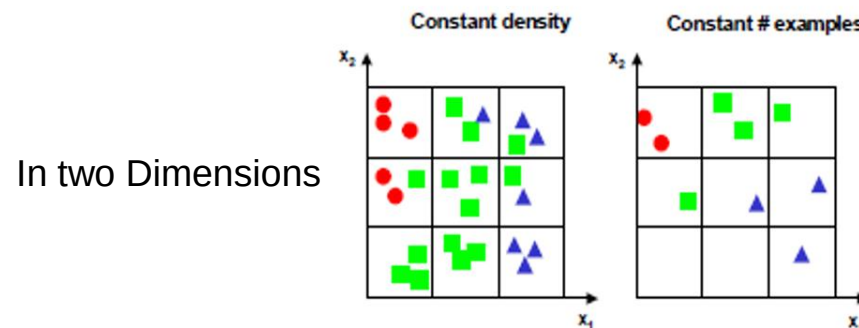
- Adding extra dimensions to a data space will exponentially increase the volume of data space.

Computation and Storage Costs:

- Algorithms often have higher time and space complexities in higher dimensions.

The Curse of Dimensionality

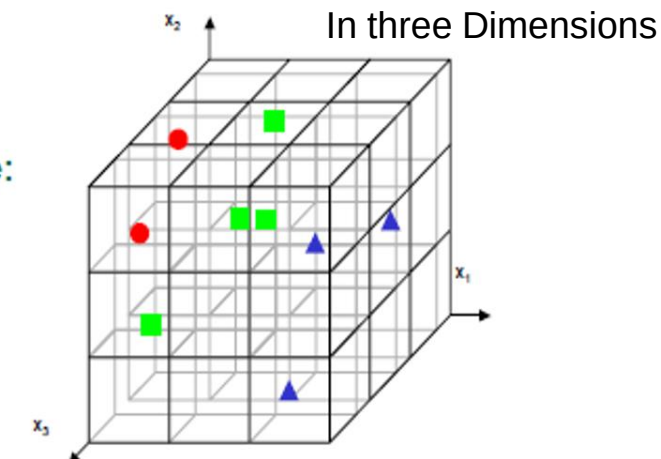
- We decide to preserve the granularity of each axis, which raises the number of bins from 3 (in 1D) to $3^2=9$ (in 2D)
 - At this point we are faced with a decision: do we maintain the density of examples per bin or do we keep the number of examples we used for the one-dimensional case?
 - Choosing to maintain the density increases the number of examples from 9 (in 1D) to 27 (in 2D)
 - Choosing to maintain the number of examples results in a 2D scatter plot that is very sparse



In one Dimension



- Moving to three features makes the problem worse:
 - The number of bins grows to $3^3=27$
 - For the same density of examples the number of needed examples becomes 81
 - For the same number of examples, well, the 3D scatter plot is almost empty



Break

Dr. Sen Wang left me a task...

`eval.uq.edu.au`

Outline

- **Big Data** Applications
 - Connecting Dots: **From Small to Big**
 - Discovering Specifics: **From Big to Small**
 - Inferencing: **Knowing Unknown**
 - **Dimensionality: Blessing or Curse?**
- **Effective Storage** of Big Data
 - **SQL or NoSQL**
 - **Vector database**

NoSQL

- Applications are going beyond the traditional DBMS for various reasons
- NoSQL is an umbrella term for databases /data stores which do not comply with traditional DBMS principles
 - Often in the context of big data problems
- Non SQL or Not Only SQL?



NoSQL

- **Cloud Platform:** - a viable alternative to relational databases operating on **cluster servers**
- **No Schema:** - **different types of data** is collected, stored, accessed without a Schema
- **Data Fusion:** - a data integration technique for **multi-source data**
- **Flexible Access:** - A **query model** accessing data without using traditional SQL

NoSQL

Document databases

- Typically store documents as json blobs (we will look at these shortly)
- Different data fields and their values can be indexed and used for retrieval

Key-value stores

- Basically persistent hash tables; all access is done via keys, accessing values

Wide-column stores

- Store data in tables, rows, and dynamic columns; like RDBMS but names and data types can differ among rows of the same table (!!)

Graph databases

- Store data pertaining to entities or objects (nodes) and the relationships among them (edges)
- Basically a large persistent “graph” structure

NoSQL: Document Databases

- Typically, “everything you need” will be stored together in a single “blob” called a document.
- Use Case: General purpose, most similar to RDBMS.
- Support “CRUD”:
 - Create: add data to the store
 - Read: Get a document based on a key or other value
 - Update: Update part or all of some document
 - Delete: Remove a document from the store

NoSQL: JSON Documents

Many NoSQL engines use “JavaScript Object Notation” (JSON) as the de-facto document storage/querying mechanism

```
{  
    "studentID": "ZhiChen123",  
    "firstName": "Zhi",  
    "age": "30",  
    "program": "Master of Data Science",  
    "courses": ['INFS3200', 'INFS3208'],  
}
```

NoSQL Language

To store objects using **key-values**.

To add/delete/query massive arrays and **still allow for persistence** and fault tolerance.

To implement large data query on **MapReduce** framework

E.g., an SQL code:

SELECT * FROM users WHERE age > 10

can be **implemented** by a Javascript function that runs against every item in the database:

```
function (doc)
{
  if (doc.objType == "users") {
    if (doc.age > 10) {
      emit(doc._id, null)
    }
  }
}
```

NoSQL: Key-value stores

Perhaps the most “simple” NoSQL database

- Essentially a persistent, in-memory, hash table
- All access is done via a “key”
 - **Keys must be unique**
 - Can be stored as ‘String’ data type of convenience and flexibility.

Supports the typical CRUD operations discussed previously (updates via “Overwrite”)

Keys	Values
1	ABC, @!
2	AAA, 01/01/2022
3	123, AAA
4	8, YUIB, 582
5	AAA, DDD

NoSQL: Key-value stores

Use cases:

- High volumes of small reads and writes and infrequent updates
- Often used for caching items
 - Shopping carts
 - Webpages
 - Images
 - ...

NoSQL: Graph Database

- Typically Index “RDF triples”
 - RDF: “Resource Description Framework” – the building block of the Semantic Web
- RDF triples are used to encode data in a **semi-structured** way
 - <Subject><Predicate><Object>
- Use cases:
 - Social Media Network, Recommender Systems, ...
 - Not that suitable for applications that primarily deal with data that can be easily represented in a traditional relational database (e.g., Columnar data)

NoSQL Systems

CouchDB - a document-oriented database that can be queried and indexed in a MapReduce fashion using JavaScript. CouchDB also offers incremental replication with bi-directional conflict detection and resolution.

MongoDB - a scalable, high-performance, open source, document-oriented database system.

Hadoop and HBase - The Apache Hadoop project develops open-source software for reliable, scalable, distributed computing. HBase is the Hadoop database system. It supports random, real-time read/write access to “Big Data.”



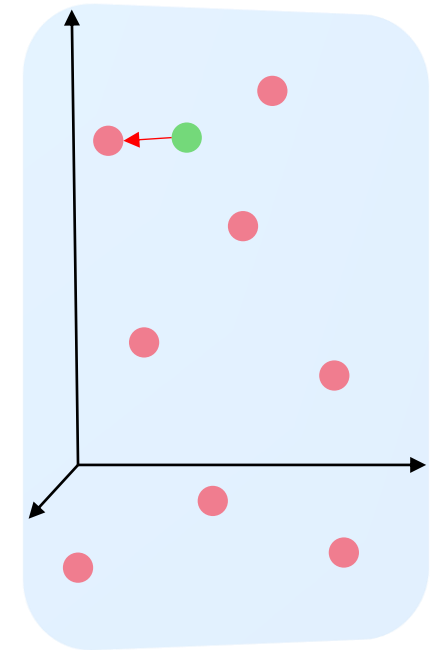
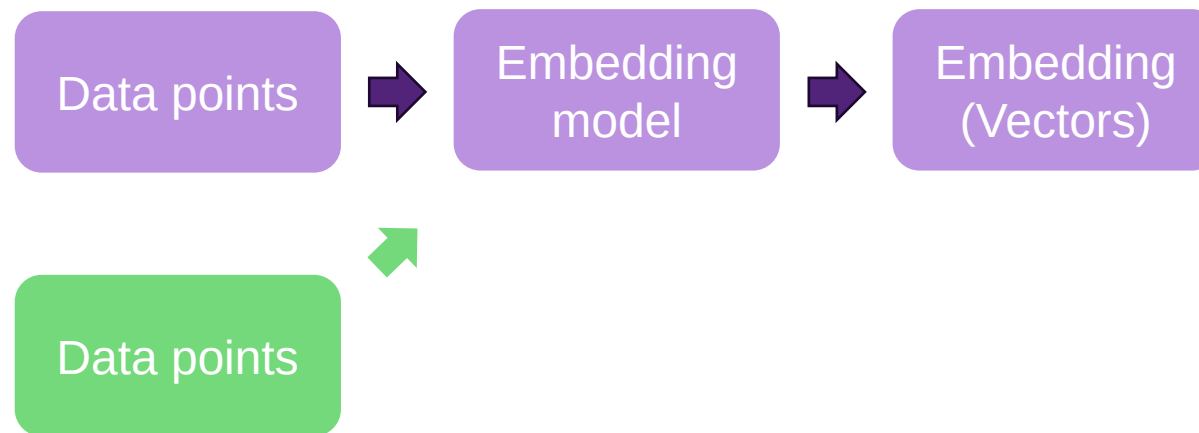
NoSQL Summary

- Targeted at 'big data' processing regimes
- Vastly different to relational database management systems
 - Often cloud/cluster-based
 - Easy to scale
 - No schemas
 - Different storage/retrieval capabilities
 - Often less strict about things like ACID properties
- Often tailor to specific use-cases; many offerings available

Vector database

Specialized database system designed to store, manage, and search high-dimensional vectors.

- Some key concepts:
 - Embedding model
 - Embedding
 - Data
 - Query



Vector database

Specialized database system designed to store, manage, and search high-dimensional vectors.

Some key concepts and questions:

- Embedding model
 - What could be an embedding model?
 - Language model? Vision model? Multi-modality?
- Embedding
- Data
- Query

Vector database

Specialized database system designed to store, manage, and search high-dimensional vectors.

Some key concepts and questions:

- Embedding model
- Embedding
 - What should be the length of the embedding?
- Data
- Query

Vector database

Specialized database system designed to store, manage, and search high-dimensional vectors.

Some key concepts and questions:

- Embedding model
- Embedding
- Data
 - Which type of data is suitable for vector database?
 - Unstructured, may require vector-level comparison.
- Query

Vector database

Specialized database system designed to store, manage, and search high-dimensional vectors.

Some key concepts and questions:

- Embedding model
- Embedding
- Data
- Query
 - What if the query is “out-of-distribution”?

Vector Database: Recommender System

Consider online shopping recommendation as a typical use case of vector database

Objective: Help users to discover new items based on their -

- Preferences
- Webpage interaction
- Liked items
- Purchase history
- ...

	 View List	 Cart List	 Purchase List
 Rachel	  	  	  
 Susan	  	 	 
 Evan	  	  	  

Vector Database: Recommender System

Similarity score between user vectors and item vectors

- Higher score (?) → more likely to be recommended

System also needs to handle **updates**:

- User / Item
- Could be a million-scale

Existing DBMSs were not built to **manage vector** data on that **scale**.

Underlying data management requirements of their applications differ vector-driven applications.

- E.g., JOIN is not really required in most of the vector driven scenario

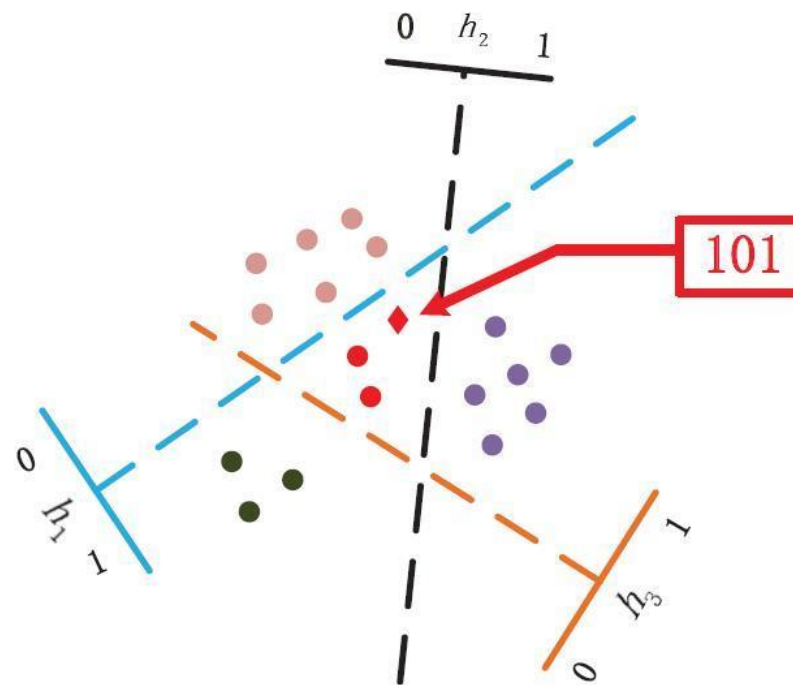
Approximate Nearest Neighbour Search

Locality Sensitive Hashing (LSH)

- Similar items will be hashed to the same "buckets" with high probability, whereas dissimilar items will be hashed to different buckets with high probability.
- **Hash Functions:** multiple hash functions are designed so that similar items collide
 - Items which are similar according to some similarity are likely to end up in the same hash bucket
- **Buckets:** The output of the hash functions is used to partition the data into different buckets or bins.

Approximate Nearest Neighbour Search

Locality Sensitive Hashing (LSH)



Exact or Approx.?: Nearest Neighbor Search

Trade-off: Time-efficiency vs. Accuracy

- Continues or discrete?
- All data points or representative data points?

What's Next

This week's Tutorial: Consultation

Next week's Lecture: Revision